

۱. طراحی و پیاده سازی پردازنده ی تک چرخه ی ۳۲ بیتی

هدف از این تمرین، طراحی و پیاده سازی یک پردازنده ی تک چرخه ^۱ کامل با قابلیت اجرای دستورات محاسباتی، انتقال داده و پرش است. شما یک هسته ی پردازنده ی ۳۲ بیتی را در محیط Logisim و یا به صورت کد به زبان Verilog پیاده سازی خواهید کرد که می تواند مجموعه دستورات مشخصی را اجرا کند.

مشخصات ورودی ها و خروجی ها

ورودی ها:

- Clk: سیگنال ساعت ^۲
- Rst: سیگنال بازنشانی ^۳
- InstrIn: دستور خوانده شده (۳۲ بیت)
- DataIn: داده خوانده شده (۳۲ بیت)

خروجی ها:

- R0, R2, ..., R31: ۳۲ ثبات ۳۲ بیتی
- InstrAddr: نشانی دستور (۳۲ بیت - مضرب ۴)
- DataAddr: نشانی داده (۳۲ بیت - مضرب ۴)
- DataWrite: نوشتن یا خواندن از حافظه داده
- DataOut: داده در حال نوشته شدن به حافظه (۳۲ بیت)

مجموعه دستورات ISA

پردازنده ی شما باید دستورات زیر را پشتیبانی کند. قالب کلی دستورات از معماری MIPS الهام گرفته شده است.

¹single-cycle

²Clock

³Reset

● قالب کلی دستور نوع R-type

قالب کلی این دستورات مطابق جدول ۱ است.

جدول ۱: قالب کلی دستور نوع R-type

31-26	25-21	20-16	15-11	10-6	5-0
opcode	rs	rt	rd	shamt	funct
6	5	5	5	5	6

کد عملیات opcode برای تمامی آن‌ها ۰۰۰۰۰۰ است و نوع عملیات توسط فیلد funct مشخص می‌شود. عملیات مربوط به این قالب در جدول ۲ آمده است.

جدول ۲: دستورات محاسباتی نوع R-type

دستور	ریشه	شرح	funct
ADD \$rd, \$rs, \$rt	R	$\$rd = \$rs + \$rt$	100000
SUB \$rd, \$rs, \$rt	R	$\$rd = \$rs - \$rt$	100010
MUL \$rd, \$rs, \$rt	R	$\$rd = (\$rs * \$rt)[31:0]$	011000
DIV \$rd, \$rs, \$rt	R	$\$rd = \$rs / \$rt$	011010
JR \$rs	R	$PC = \$rs$	001000

● قالب کلی دستور نوع I-type

قالب کلی این دستورات مطابق جدول ۳ است.

جدول ۳: قالب کلی دستور نوع I-type

31-26	25-21	20-16	15-0
opcode	rs	rt	immediate
6	5	5	16

عملیات مربوط به دستورات انتقال داده که از قالب I استفاده می‌کنند، در جدول ۴ آمده است.

جدول ۴: دستورات انتقال داده نوع I-type

دستور	opcode	ریشه	شرح
LW \$rt, imm(\$rs)	100011	I	$\$rt = \text{Mem}[\$rs + \text{sign_ext}(\text{imm})]$
SW \$rt, imm(\$rs)	101011	I	$\text{Mem}[\$rs + \text{sign_ext}(\text{imm})] = \rt

دستورات محاسباتی مقدار ثابت از قالب I-type استفاده می‌کنند. در این دستورات، یکی از عملوندها از ثبات rs و عملوند دیگر از مقدار ثابت گرفته می‌شود. عملیات‌های مربوط به این دستورات در جدول ۵ آمده است.

جدول ۵: دستورات محاسباتی مقدار ثابت نوع I-type

دستور	opcode	ریشه	شرح
ADDI \$rt, \$rs, imm	001000	I	$\$rt = \$rs + \text{sign_ext}(\text{imm})$
SUBI \$rt, \$rs, imm	001001	I	$\$rt = \$rs - \text{sign_ext}(\text{imm})$
LUI \$rt, imm	001111	I	$\$rt = \{\text{imm}, 16'b0\}$

• قالب کلی دستور نوع J-type

قالب کلی این دستورات مطابق جدول ۶ است.

جدول ۶: قالب کلی دستور نوع J-type

31-26	25-0
opcode	address
6	26

عملیات مربوط به دستورات پرش در جدول ۷ آمده است. دستورهایی J و JAL از قالب J و دستور BEQ از قالب I استفاده می‌کند.

جدول ۷: دستورات پرش نوع J-type و I-type

دستور	opcode	ریشه	شرح
J address	000010	J	$PC = \{PC[31:28], \text{address}, 2'b00\}$
JAL address	000011	J	$\$31 = PC+4;$ $PC = \{PC[31:28], \text{address}, 2'b00\}$
BEQ \$rs, \$rt, label	000100	I	if \$rs == \$rt then $PC = PC+4 + \text{sign_ext}(\text{imm}) \ll 2$

منطق پیاده سازی

دسترسی در حافظه به کلمات ۴ بایتی است.

خواندن دستورات

برای خواندن دستورات، کافی است نشانی مورد نظر خود را به InstrAddr متصل کنید، سپس دستور قرار گرفته در نشانی درخواستی را در همان چرخه ساعت از InstrIn دریافت کنید. همینطور به این موضوع توجه کنید که PC همیشه باید مضرب ۴ باشد و در هنگام نشانی دهی به حافظه، دو ۰ سمت راست نشانی را باید نادیده بگیرید.

ارتباط با حافظه داده

برای خواندن از حافظه، کافی است DataWrite را به ۰ و DataAddr را به نشانی مورد نظرتان متصل کرده و داده قرار گرفته در آن محل را در همان چرخه ساعت از DataIn بخوانید. همچنین برای نوشتن به حافظه، کافی است DataWrite

را به ۱ و DataAddr را به نشانی مورد نظر و DataOut را به داده مورد نظر برای نوشتن متصل کنید تا در لبه بالا رونده ساعت، عملیات مورد نظر انجام شود. توجه کنید که درخواست های این حافظه نیز مانند حافظه دستورات به صورت مضرب ۴ است و شما باید هنگام نشانی دهی به حافظه، دو ۰ سمت راست نشانی را نادیده بگیرید.

معماری کلی پردازنده

پردازنده شما به صورت تک چرخه طراحی می شود. بنابراین طول چرخه ساعت باید برای اجرای کامل هر دستور کافی باشد. معماری کلی پردازنده شامل بلوک های اصلی مسیر داده و سیگنال های کنترلی لازم برای اجرای دستورات است.

بلوک های اصلی مسیر داده

پردازنده شما باید از بخش های زیر تشکیل شود:

- **ثبات شمارنده برنامه^۴:** ثبات ۳۲ بیتی که نشانی دستور بعدی را نگه می دارد. مقدار PC+4 برای رفتن به دستور بعدی و همچنین برای ذخیره نشانی بازگشت در دستور JAL استفاده می شود.
- **بانک ثبات^۵:** ۳۲ ثبات ۳۲ بیتی با دو درگاه خواندن و یک درگاه نوشتن. ثبات ۰ همواره باید مقدار صفر داشته باشد، بنابراین با استفاده از یک مالتی پلکسر روی ورودی نوشتن بانک ثبات، از نوشته شدن هر مقداری در ثبات ۰ جلوگیری کنید.
- **واحد حساب و منطق^۶:** واحدی که عملیات جمع، تفریق، ضرب، تقسیم و مقایسه لازم برای دستور BEQ را انجام دهد. می توانید از پیمانه های تمارین قبلی استفاده کنید.
- **واحد کنترل^۷:** مدار ترکیبی که با دریافت opcode و funct، سیگنال های کنترلی شامل RegWrite، RegDst، ALUSrc، MemRead، MemWrite، Branch، Jump، ALUOp، Jal و ... را تولید می کند.
- **جمع کننده ها و شیفتهنده ها:** برای محاسبه PC+4 و نشانی مقصد پرش های شرطی. در پرش نوع BEQ، شیفته ۲ بیتی به چپ روی مقدار ثابت، که ۱۶ بیت است، اعمال کنید و نتیجه را با PC+4 جمع کنید.

سیگنال های کنترلی

برای اجرای صحیح دستورات، واحد کنترل باید حداقل سیگنال های زیر را تولید کند:

- **RegDst:** انتخاب ثبات مقصد، rd برای دستورهای نوع R-type، rt برای دستورهای نوع I-type، و ۳۱\$ برای دستور JAL.
- **RegWrite:** اجازه ی نوشتن در بانک ثبات. مقدار ۱ نشان دهنده ی فعال بودن عملیات نوشتن است.
- **ALUSrc:** انتخاب عملوند دوم واحد حساب و منطق. مقدار ۰ به معنی انتخاب ثبات rt و مقدار ۱ به معنی انتخاب مقدار ثابت است.
- **MemtoReg:** انتخاب داده ی ورودی بانک ثبات. مقدار ۰ خروجی واحد حساب و منطق، مقدار ۱ خروجی حافظه، و مقدار ۲ مقدار PC+4 را برای دستور JAL انتخاب می کند.
- **MemRead:** فعال سازی خواندن از حافظه داده.
- **MemWrite:** فعال سازی نوشتن در حافظه داده.

^۴PC

^۵Register File

^۶ALU

^۷Control Unit

- **Branch**: نشان‌دهنده‌ی دستور BEQ که همراه با سیگنال Zero برای تعیین انجام پرش استفاده می‌شود.
- **Jump**: نشان‌دهنده‌ی دستور J یا JAL.
- **Jal**: نشان‌دهنده‌ی دستور JAL که برای ذخیره‌ی مقدار PC+4 در ثبات \$31 استفاده می‌شود.
- **ALUOp**: چند بیت کنترلی برای انتخاب عملیات واحد حساب و منطق.

نکته: توجه کنید که اجباری به جدا سازی واحد کنترل از مسیر داده (مانند مدارات ترتیبی کلاسیک) نیست.

نحوه آزمون

پردازنده شما توسط کد زیر مورد ارزیابی قرار می‌گیرد. در ابتدا مدار در وضعیت بازنشانی قرار می‌گیرد، سپس درون حافظه مربوطه، دستورات مورد نظر قرار گرفته و حافظه داده * می‌شود. در نهایت مدار از وضعیت بازنشانی خارج شده و پردازنده باید از همان چرخه ساعت شروع به کار کند. از این لحظه وضعیت ثبات‌های پردازنده شما با یک پردازنده معیار مقایسه شده و نمره دهی می‌شود.

```
#####
# MAIN PROGRAM
#####

LUI    $26,1          # $26 = 0x00010000

LW     $1,0($26)       # n = Mem[0x00010000] = 10
JAL    fib
ADD    $20,$2,$0       # save fib(10)

LW     $1,4($26)       # n = Mem[0x00010004] = 11
JAL    fib
ADD    $21,$2,$0       # save fib(11)

MUL    $22,$20,$21     # product = fib(10)*fib(11)

ADD    $1,$22,$0       # input to base3 converter
JAL    to_base3

J      finish

#####
# FIBONACCI SUBROUTINE
#####

fib:
BEQ    $1,$0,fib_zero

ADDI   $3,$0,0         # a = 0
ADDI   $4,$0,1         # b = 1
ADDI   $5,$0,1         # i = 1
```

```

fib_loop:
    BEQ    $5,$1,fib_done

    ADD    $6,$3,$4
    ADD    $3,$4,$0
    ADD    $4,$6,$0

    ADDI   $5,$5,1
    J      fib_loop

fib_done:
    ADD    $2,$4,$0
    JR     $31

fib_zero:
    ADDI   $2,$0,0
    JR     $31

#####
# BASE-3 CONVERSION
#####

to_base3:
    ADDI   $10,$26,100    # output address = 0x00010064
    ADDI   $11,$0,3       # divisor = 3

base3_loop:
    BEQ    $1,$0,base3_done

    DIV    $12,$1,$11     # quotient = n/3
    MUL    $13,$12,$11
    SUB    $14,$1,$13     # remainder

    SW     $14,0($10)

    ADDI   $10,$10,4
    ADD    $1,$12,$0

    J      base3_loop

base3_done:
    JR     $31

#####
# END
#####

```

finish :

ADDI \$30 , \$0 , 123

خروجی‌های مورد انتظار برای تحویل

- فایل طراحی مدار در محیط Logisim (نام فایل: HW5-STUDENT_ID.circ) یا کد پیاده‌سازی مدار به زبان Verilog. توجه کنید که اجازه استفاده از عملگرهای ضرب و تقسیم که به صورت ترکیبی کار می‌کنند را ندارید.
- گزارش کاملی از نحوه طراحی و اجرای آزمون ارائه کنید (نام گزارش: HW5-STUDENT_ID.pdf). توجه داشته باشید که نمره‌دهی تنها بر اساس نتایج همین آزمون انجام می‌شود.